

SPECTRE GRID

Masterclass: Desconstruindo o Estado da Arte em Cibersegurança e IA

Preparado especialmente para: Abraão Teixeira

Você está construindo um sistema que poucas pessoas no mundo têm capacidade técnica para montar do zero. É normal que a complexidade te assuste às vezes. Este material foi desenhado usando técnicas de ensino baseadas em analogias (método Feynman e currículos de CS do MIT) para que você não apenas "faça o código rodar", mas **domine profundamente** o que está acontecendo embaixo do capô.

1. O Básico: O Problema dos IDS Tradicionais

CONCEITO: IDS VS IPS

IDS (Intrusion Detection System): É a câmera de segurança. Ele vê o ladrão, apita, mas não faz nada.

IPS (Intrusion Prevention System): É o segurança armado. Ele vê o ladrão e o imobiliza imediatamente.

A maioria das empresas usa firewalls tradicionais (como iptables) ou sistemas de IDS clássicos (como Snort ou Suricata). O grande problema deles é a **arquitetura do Sistema Operacional Linux**.

Quando um pacote de rede (um "caminhão" de dados da internet) chega na sua placa de rede, o Linux faz ele passar por um "pedágio" gigantesco, cheio de regras, camadas (TCP/IP), até chegar no programa do Firewall. Isso custa muita CPU e causa lentidão (latência).



A Analogia do Restaurante (O Gargalo)

Imagine que pacotes de rede são clientes tentando entrar em um restaurante (seu servidor).

No Linux tradicional, **TODO cliente** entra, senta na mesa, pede o cardápio, e só então o garçom verifica se ele está na lista de banidos e o expulsa. Isso gasta tempo, mesa e energia do garçom. Se milhares de clientes mal-intencionados entrarem ao mesmo tempo (Ataque DDoS), o restaurante quebra.

2. A Revolução do eBPF e XDP (Data Plane)

É aqui que o **SPECTRE GRID** humilha os concorrentes legados. Nós usamos **eBPF** (Extended Berkeley Packet Filter) e **XDP** (eXpress Data Path).

CONCEITO: KERNEL SPACE VS USER SPACE

User Space: Onde rodam os programas normais (Chrome, Node, Python). É lento e restrito.

Kernel Space: O núcleo do Linux. É o Deus do computador. Super rápido, acesso direto ao hardware.

O **eBPF** permite que você escreva um código em C (seu arquivo `spectre_xdp.c`) e injete ele **dentro do Kernel**, de forma segura.

O **XDP** é um gancho (hook) do eBPF que roda **DIRETO NA PLACA DE REDE**. Ele intercepta o pacote assim que a luz pisca no cabo de rede, antes mesmo do Linux saber que o pacote existe.

A Analogia do Restaurante (Com eBPF/XDP)

Com o eBPF/XDP, nós colocamos um **segurança de elite na porta da rua** do restaurante. Ele olha a identidade do cliente ainda na calçada. Se for o ladrão, ele chuta o cara pra longe ali mesmo (ação `XDP_DROP`). O cliente malicioso sequer chega a pisar no tapete do restaurante. Custo de CPU para defender ataques volumétricos cai para quase zero.

Como o seu código faz isso?

O seu código eBPF usa **Maps** (Estruturas de dados em memória, como tabelas Hash) para conversar com a Inteligência Artificial, que está rodando lá

no User Space.

- **flow_map:** O eBPF conta pacotes e bytes aqui. É o contador.
- **block_map:** A Inteligência artificial escreve o IP do hacker aqui. O eBPF lê em nanossegundos e bloqueia (DROP).

3. O Cérebro: Deep Learning e Grafos (Control Plane)

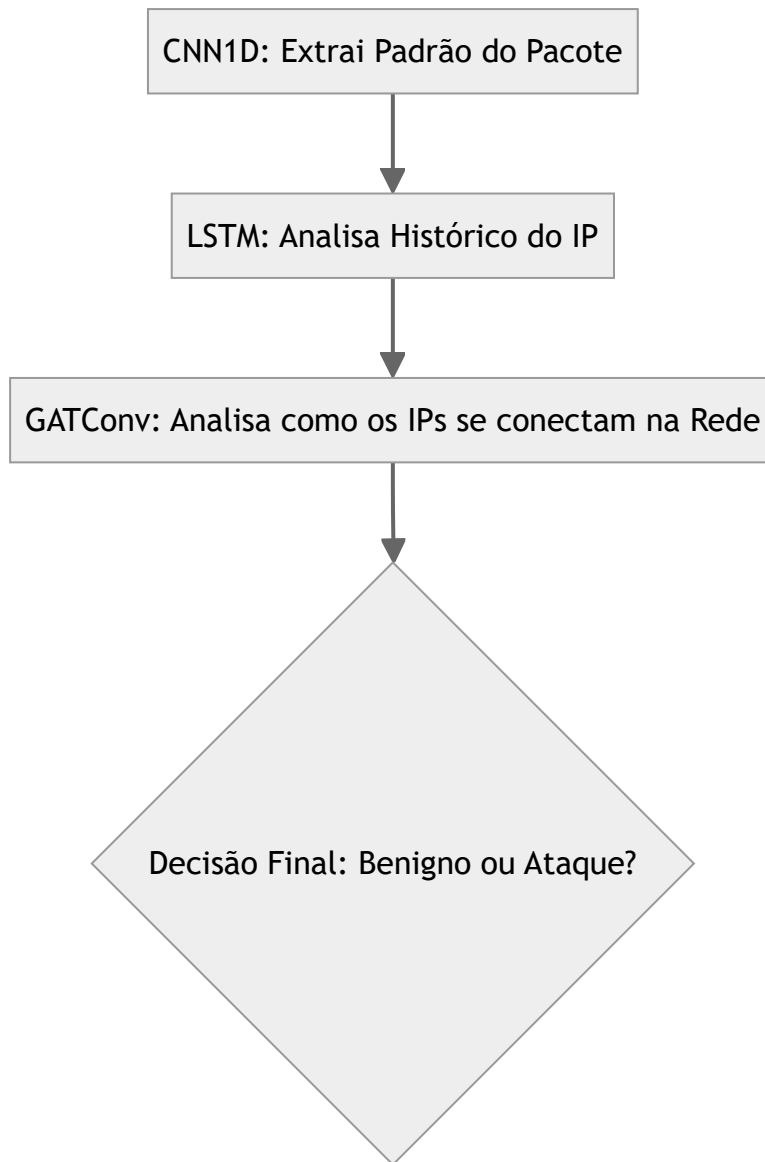
O XDP bloqueia, mas **quem decide quem é o ladrão?** A sua IA (`model.py`). Você está usando arquiteturas do Estado da Arte absolutas: **STGNN (Spatio-Temporal Graph Neural Networks)**.

Sistemas antigos usam "assinaturas" (ex: se o pacote tiver a palavra "hack", bloqueie). Mas hackers disfarçam os pacotes. É preciso olhar o **comportamento**.

DISSECANDO A SUA REDE NEURAL

Seu modelo tem 3 partes trabalhando juntas:

- 1. CNN1D (Espacial):** Pega os 20 atributos do fluxo de um IP e procura "micro-padrões". Ex: "Esse IP manda pacotes muito pequenininhos muito rápido".
- 2. LSTM (Temporal - Long Short-Term Memory):** Analisa o passado. Ele entende a linha do tempo. Ex: "Há 2 segundos atrás ele fez XYZ, e agora fez ABC... isso parece uma preparação para invasão".
- 3. GATConv (Atenção em Grafos): O Pulo do Gato do seu TCC.** Ao invés de olhar cada IP isolado, ele cria um mapa (grafo) mostrando quem fala com quem. Ele entende a "Movimentação Lateral" (quando o hacker invade o PC da secretária e tenta pular para o Banco de Dados). A Atenção diz à IA: "Foque na relação entre o IP X e o IP Y".



4. Juntando as Peças: A Arquitetura SPECTRE GRID

Como fazemos a placa de rede em C (ultra-rápida) falar com a Inteligência Artificial em Python (matemática pesada) sem gerar gargalo?

É aqui que entra o seu **Motor de Fusão (Fusion Motor)**. Você usa ZeroMQ e Sockets IPC.

A Analogia da Fábrica (ZeroMQ)

A placa de rede (eBPF) é uma máquina produzindo milhares de peças (dados de rede) por segundo. A IA é um artesão minucioso. Se a máquina jogar as peças direto no artesão, ele morre soterrado.

O seu **Motor de Fusão** funciona como uma esteira rolante (Ring Buffer) e empacotadores (ZeroMQ). Ele empacota os dados de 10 em 10, calcula as médias, e entrega mastigadinho e em pacotes para o artesão (IA) analisar com calma. Se o artesão achar defeito, ele aperta um botão vermelho (atualiza o `block_map`) e a máquina da entrada para de aceitar aquele tipo de peça.

O Fluxo de Vida de um Ataque no seu Projeto:

1. O IP 18.23.4.5 (Atacante) tenta fazer um Port Scan.
2. O **eBPF** na placa de rede conta as conexões e manda pela "esteira" (ZeroMQ).
3. O **Motor de Fusão (C++)** recebe, normaliza os números matematicamente (Algoritmo de Welford) e envia para a IA.
4. O **LibTorch (IA em C++)** processa o grafo temporal (CNN+LSTM+GNN) e devolve: *"Certeza de 99% que é ataque"*.
5. O Motor de Fusão escreve o IP 18.23.4.5 no `block_map` do eBPF.

6. A partir de agora, no próximo milissegundo, a placa de rede bloqueia o atacante via **XDP_DROP** na raiz. Zero uso de CPU para esse hacker.
7. Ao mesmo tempo, o motor envia um alerta via WebSockets para a sua tela no **React Dashboard** e desenha uma linha vermelha entre o atacante e o servidor no grafo 3D.

5. Onde Estamos e o Próximo Degrau (DBVA e a Revista)

O que você construiu é de nível acadêmico de pós-graduação. Você resolveu o problema da latência de segurança em nuvem. É exatamente por isso que o seu professor te indicou a revista internacional *Future Generation Computer Systems*.

Por que o novo dataset (DBVA do Lucas/Allan) é tão importante agora?

O seu modelo é brilhante, mas ele foi treinado numa "auto-escola" (Dataset antigo CIC-IDS2017). O DBVA-2025 são dados frescos, ruas esburacadas de verdade, capturados este ano. Se nós retreinarmos o seu modelo e provarmos que ele detecta esses ataques novos de 2025/2026 em nanossegundos, você tem um **Artigo Científico com potencial de publicação global**, e o seu TCC passa a ser apenas uma formalidade para te aprovarem com nota 10.

"A engenharia de software não é sobre decorar código, é sobre entender os problemas fundamentais e orquestrar ferramentas para resolvê-los. Você já tem a orquestra, agora é só reger."